But how did Hotwire Native know to fetch a new version of this page from the server? With some clever use of the *snapshot cache*.

The Hotwire Native Snapshot Cache

Hotwire Native employs a few tricks to make apps feel more like native ones. One of those is the snapshot cache, which ensures previously fetched pages appear to load instantly. Try it out with the iOS app you're creating. Navigate to a page, then tap the back button. Notice how the content is rendered right away, without a spinner or delay?

This is because a screenshot is taken of every single page. When you navigate from one page to another, the snapshot of the previous page is written to the cache. When a user navigates back, the library loads this image from memory, so it appears like the content loads instantly.

Under the hood, Hotwire Native is moving a single web view between view controllers. Web views take a long time to spin up, so sharing one between screens further increases the perceived speed of the app. The snapshot cache gives users a near-native experience when navigating forward and backward around the app.

But as with all caches, they eventually need to be busted. If not, you might end up displaying stale or out-of-date content to your users.

Clearing the snapshot cache ensures that tapping the back button makes a fresh request to the server, populating the screen with the latest data. Every time the app performs a non-GET request, like when submitting a form, Hotwire Native will bust the cache.

With iOS path configuration wired up, let's shift gears to Android.